

Лабораторна робота №4

Розробка вебсерверів засобами фреймворку ASP.NET.

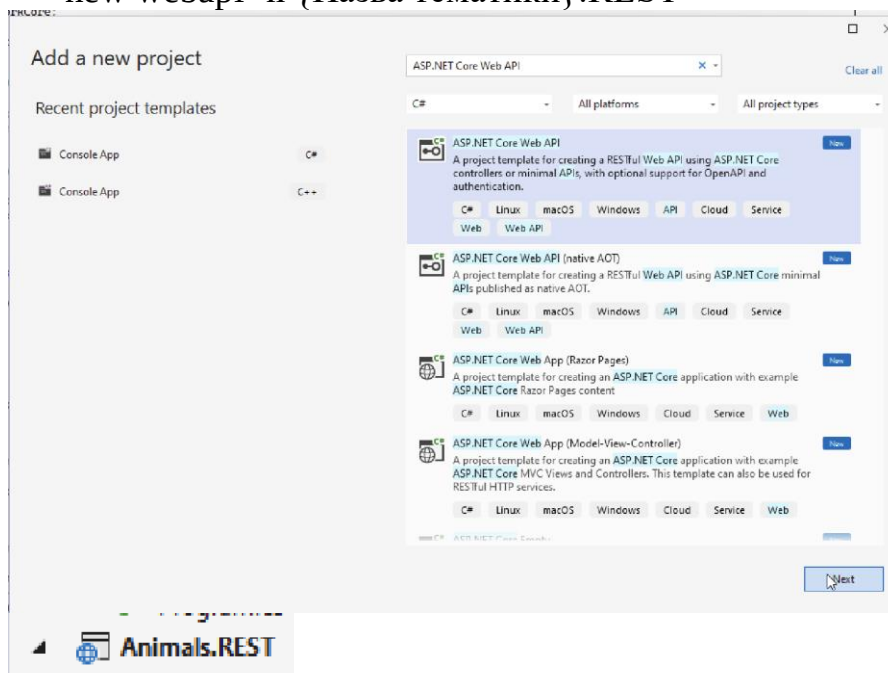
Виконав студент групи 302-ТК

Рогочий Михайло

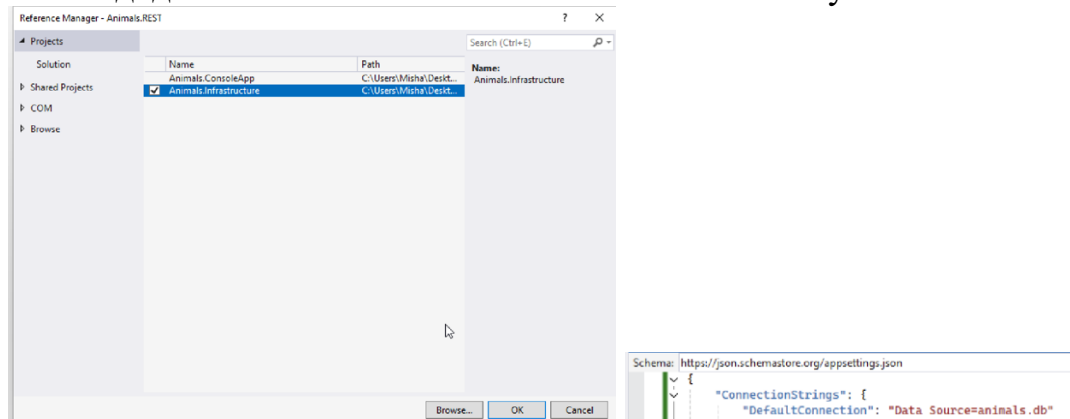
Передумова:

Завдання

1. Створити проєкт за шаблоном [ASP.NET](#) Core Web API {Назва тематики}.REST використовуючи Visual Studio або .NET SDK: `dotnet new webapi -n {Назва тематики}.REST`

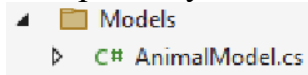


Також додаємо посилання на Animals.Infrastructure у Animals.REST



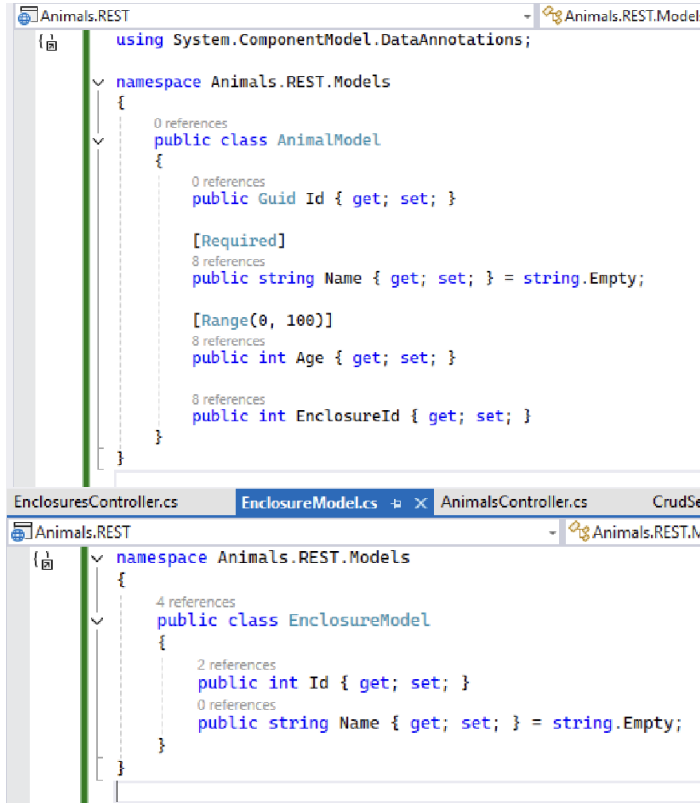
+ налаштовуємо доступ до бази даних

- Створити папку Models, у якій будуть зберігатися моделі, які будуть використовуватися контролерами у проєкті {Назва тематики}.REST.



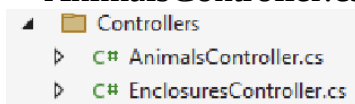
створюємо у цій папці клас AnimalModel.cs та клас EnclosureModel.cs

Та пишемо цей код, це буде модель, яку ми будемо передавати через API



- Створіть контролери у папці Controllers, які будуть описувати REST API для виконання CRUD операцій (створення, читання, оновлення, видалення) над моделями у вашій тематиці, які ви зберігали у базі даних у третій лабораторній, при цьому моделі, що використовуються у контролерах, повинні бути описані у проєкті {Назва тематики}.REST, щоб відповідати 3-рівневій архітектурі. Наприклад, якщо у вас є сутність Bus, що зберігається у БД, ви створите новий клас BusModel у папці Models нового проєкту, який використовуватиметься у контролерах. Також зверніть увагу, щоб моделі у контролерах, мали лишень необхідні властивості, та якщо якісь властивості зайві, для якихось із методів - створіть нову модель без цих властивостей.

Створюємо папку Controllers та додаємо контролери `AnimalsController.cs` та `EnclosuresController.cs`



Та пишемо ці коди в контролери: AnimalsController.cs

```
Animals.REST - Animals.REST.Controllers.AnimalsController
using Animals.REST.Models;
using Microsoft.AspNetCore.Mvc;
using Animals.Infrastructure.Repositories;

namespace Animals.REST.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AnimalsController : ControllerBase
    {
        private readonly ICrudServiceAsync<AnimalModel> _service;

        public AnimalsController(ICrudServiceAsync<AnimalModel> service)
        {
            _service = service;
        }

        // GET: api/animals
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var animals = await _service.ReadAllAsync();

            var result = animals.Select(a => new REST.Models.AnimalModel
            {
                Id = a.Id,
                Name = a.Name,
                Age = a.Age,
                EnclosureId = a.EnclosureId
            });

            return Ok(result);
        }

        // GET: api/animals/{id}
        [HttpGet("{id}")]
        public async Task<IActionResult> GetById(Guid id)
        {
            var animal = await _service.ReadAsync(id);
            if (animal == null)
                return NotFound();

            var result = new REST.Models.AnimalModel
            {
                Id = animal.Id,
                Name = animal.Name,
                Age = animal.Age,
                EnclosureId = animal.EnclosureId
            };

            return Ok(result);
        }

        // POST: api/animals
        [HttpPost]
        public async Task<IActionResult> Create([FromBody] REST.Models.AnimalModel model)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            var entity = new AnimalModel
            {
                Id = Guid.NewGuid(),
                Name = model.Name,
                Age = model.Age,
                EnclosureId = model.EnclosureId
            };

            await _service.CreateAsync(entity);
            await _service.SaveAsync();

            model.Id = entity.Id;

            return CreatedAtAction(nameof(GetById), new { id = model.Id }, model);
        }

        // PUT: api/animals/{id}
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(Guid id, [FromBody] REST.Models.AnimalModel model)
        {
            if (id != model.Id)
                return BadRequest();

            var existing = await _service.ReadAsync(id);
            if (existing == null)
                return NotFound();

            existing.Name = model.Name;
            existing.Age = model.Age;
            existing.EnclosureId = model.EnclosureId;

            await _service.UpdateAsync(existing);
            await _service.SaveAsync();

            return NoContent();
        }
    }
}
```

```

// DELETE: api/animals/{id}
[HttpDelete("{id}")]
0 references
public async Task<IActionResult> Delete(Guid id)
{
    var animal = await _service.ReadAsync(id);
    if (animal == null)
        return NotFound();

    await _service.RemoveAsync(animal);
    await _service.SaveAsync();

    return NoContent();
}
}

```

EnclosuresController.cs

```

EnclosuresController.cs  EnclosureModel.cs  AnimalsController.cs  ICrudServiceAsync.cs  Program.cs
Animals.REST
using Animals.Infrastructure.Repositories;
using Animals.REST.Models;
using Microsoft.AspNetCore.Mvc;

namespace Animals.REST.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 reference
    public class EnclosuresController : ControllerBase
    {
        private readonly ICrudServiceAsync<EnclosureModel> _service;

        0 references
        public EnclosuresController(ICrudServiceAsync<EnclosureModel> service)
        {
            _service = service;
        }

        [HttpGet]
        0 references
        public async Task<IActionResult> GetAll()
        {
            var list = await _service.ReadAllAsync();
            return Ok(list);
        }

        [HttpGet("{id}")]
        1 reference
        public async Task<IActionResult> Get(int id)
        {
            var enclosure = await _service.ReadAsync(id);
            if (enclosure == null)
                return NotFound();

            return Ok(enclosure);
        }

        [HttpPost]
        0 references
        public async Task<IActionResult> Create([FromBody] EnclosureModel enclosure)
        {
            await _service.CreateAsync(enclosure);
            await _service.SaveAsync();
            return CreatedAtAction(nameof(Get), new { id = enclosure.Id }, enclosure);
        }

        [HttpPut("{id}")]
        0 references
        public async Task<IActionResult> Update(int id, [FromBody] EnclosureModel model)
        {
            if (id != model.Id)
                return BadRequest();

            var found = await _service.ReadAsync(id);
            if (found == null)
                return NotFound();

            await _service.UpdateAsync(model);
            await _service.SaveAsync();
            return NoContent();
        }

        [HttpDelete("{id}")]
        0 references
        public async Task<IActionResult> Delete(int id)
        {
            var enclosure = await _service.ReadAsync(id);
            if (enclosure == null)
                return NotFound();

            await _service.RemoveAsync(enclosure);
            await _service.SaveAsync();
            return NoContent();
        }
    }
}

```

В цих кодах ми реалізуємо методи: GET, GET, POST, PUT, DELETE в обидвох контроллерах.

- При проектуванні Web API зверніть увагу, щоб створений API відповідав 4-ій умові побудови REST-додатку по Філдингу - однорідності інтерфейсу (див. 31 слайд), тобто використовувалися унікальні назви сутностей, правильні HTTP методи та поверталися правильні HTTP коди у HTTP відповідях, наприклад 404 якщо сутність не знайдена, або 201 якщо нова сутність створена.

У пункті 3 вже наявний код з реалізацією однорідності інтерфейсу за Філдингом. Надаю скріншот з коментарями, де саме реалізовується.

```
// GET: api/animals/{id}
[HttpGet("{id}")]
1 reference
public async Task<IActionResult> GetById(Guid id)
{
    var animal = await _service.ReadAsync(id);
    if (animal == null)
        return NotFound(); //404

    var result = new REST.Models.AnimalModel
    {
        Id = animal.Id,
        Name = animal.Name,
        Age = animal.Age,
        EnclosureId = animal.EnclosureId
    };

    return Ok(result); //200
}

// POST: api/animals
[HttpPost]
0 references
public async Task<IActionResult> Create([FromBody] REST.Models.AnimalModel model)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var entity = new AnimalModel
    {
        Id = Guid.NewGuid(),
        Name = model.Name,
        Age = model.Age,
        EnclosureId = model.EnclosureId
    };

    await _service.CreateAsync(entity);
    await _service.SaveAsync();

    model.Id = entity.Id;

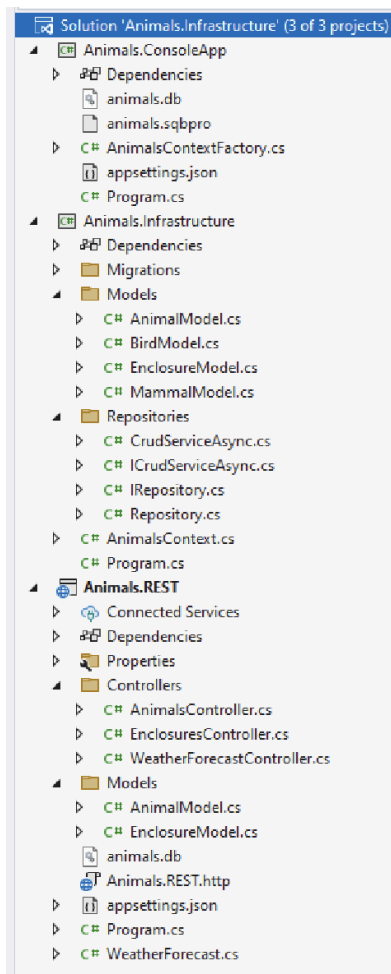
    return CreatedAtAction(nameof(GetById), new { id = model.Id }, model); //201
}
```

У REST API реалізовано принцип однорідності інтерфейсу за Філдингом: використовуються унікальні URL, коректні HTTP-методи, і API повертає правильні статуси — наприклад, 200 для успішного отримання даних, 201 при створенні, 404 якщо запис не знайдено, 204 при видаленні.

5. Використовуйте асинхрону версію дженерік CRUD сервісу, що використовує репозиторій для доступу до даних та який був реалізований у 3ій лабораторній роботі, у створених контролерах. Імплементацию CRUD сервісу, репозиторіїв та контексту, додавайте у контролери використовуючи вбудовану у ASP.NET Core підтримку впровадження залежностей(Dependency Injection).

Скріншот, де додавав посилання на Animals.Infrastructure у Animals.REST, надав у першому завданні.

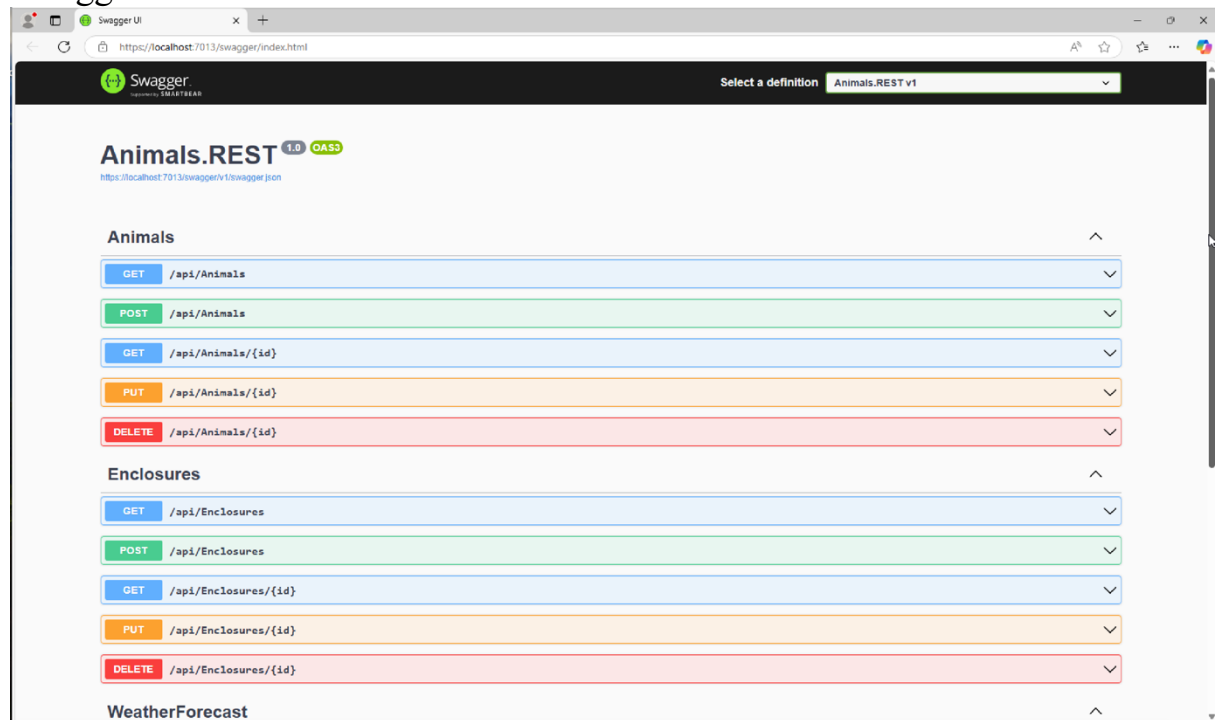
Демонструю актуальну структуру проекту з використанням 3 лабораторної роботи.



Демонстрація роботи(працездатності) застосунку:
Вивід консолі:

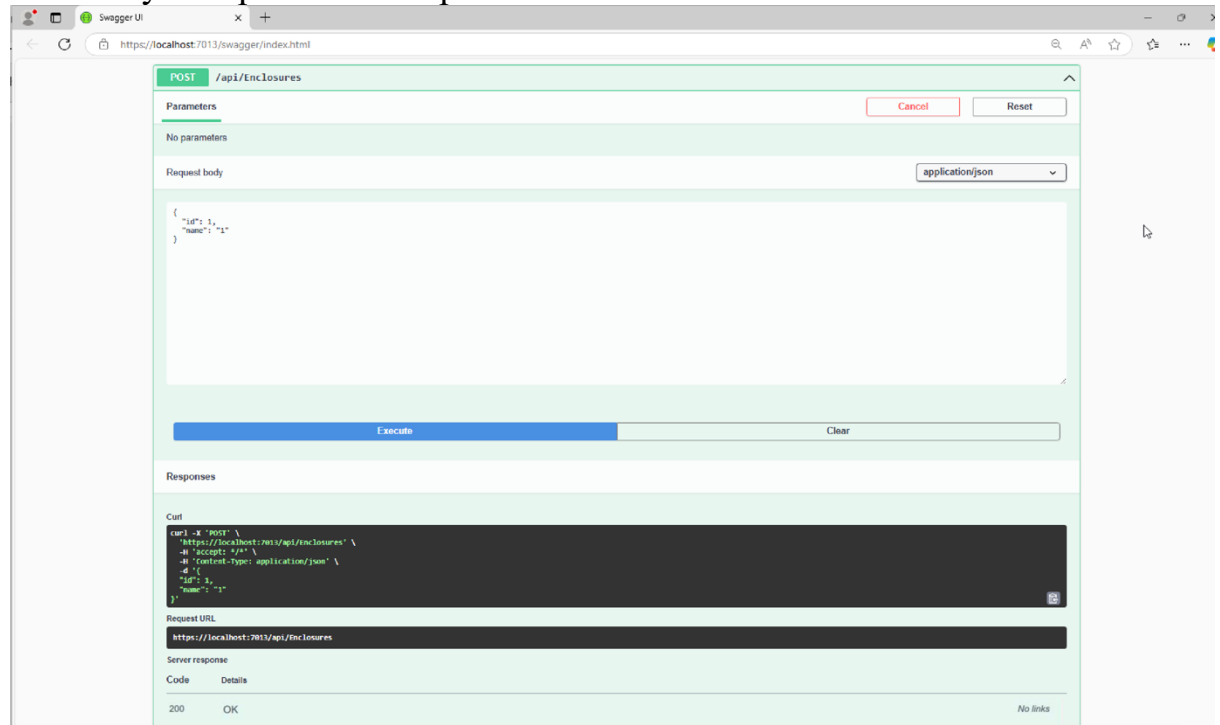
```
C:\Users\Misha\Desktop\Animals.REST\bin\Debug\net8.0\Animals.REST.exe
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7013
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5024
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\Misha\Desktop\Animals.REST
```

Swagger:

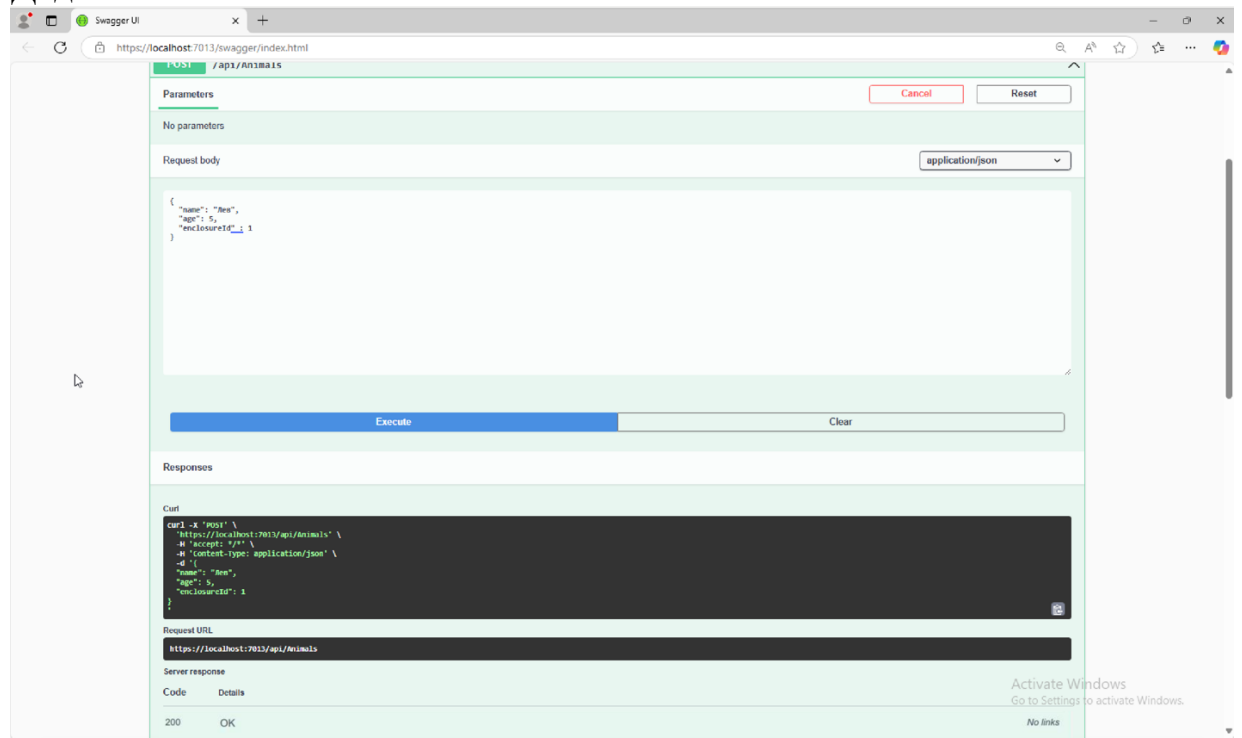


Запишемо творину через Swagger та перевіримо запис у базі даних.

Спочатку створюємо вольтер



Додаємо левів



Перевірка у базі:

